

Assignment 3: Bridge Pattern

Student	Assemgazy Olzhas
Group	SE-2528
Topic	B: Notifications
Language	Java 17
Repository	https://github.com/iuve1n/SDP_Assignment3
Base commit	5e902fb6be0726e8e82c5097667815f71f66c352
Implementation commit	4b8f3faf16ab9918e1675f817bfebd2ab66689e32
Demo result	7/7 PASS

Design

The program has two dimensions that can change independently. The abstraction side contains Notification, Reminder, and UrgentAlert. It decides how the original message is prepared. The implementation side contains Channel and its email, SMS, and push implementations. It decides how the prepared message is represented for delivery.

Bridge fits because Notification stores a Channel reference instead of creating a concrete channel. Reminder and UrgentAlert call the protected send method, which delegates to Channel.send(...). Either hierarchy can grow without adding a class for every notification and channel combination.

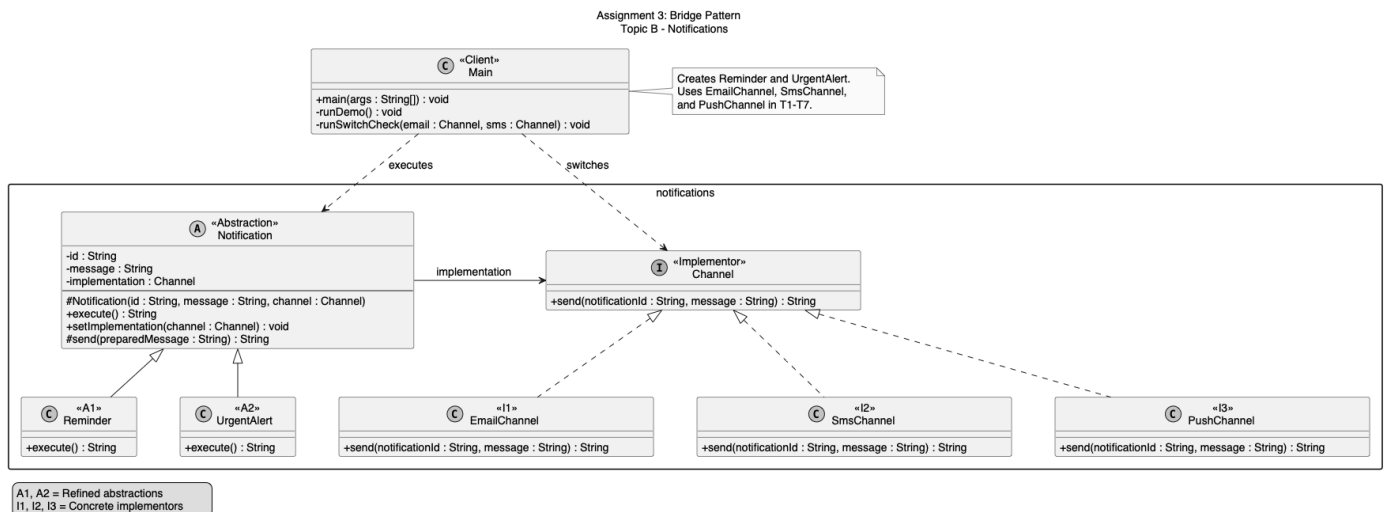


Figure 1. Final class diagram. The implementation field is typed as Channel and links the abstraction hierarchy to all three concrete implementors.

For example, Reminder.execute() adds the normal reminder prefix and calls send(). Notification.send() then calls the active Channel object. Main demonstrates all combinations and replaces the channel on one existing Reminder during T5.

Clean code examples

These excerpts are taken from the submitted source. Each example has one specific purpose in the design.

1. Separation of responsibilities

```
public interface Channel {
    String send(String notificationId, String message);
}

public abstract String execute();
```

Channel owns the delivery operation, while Notification subclasses own message preparation. A change to SMS formatting does not change Reminder or UrgentAlert.

2. Meaningful names

```
public String execute() {
    return send("Reminder: " + getMessage());
}
```

The names Reminder, execute, send, and getMessage describe the work directly. The method can be read without a comment.

3. Small focused methods

```
public String send(String notificationId, String message) {
    return "SMS [id=" + notificationId
        + ", message=" + message + "];"
}
```

SmsChannel.send() performs one task: it returns the single-line SMS representation. There is no notification-type logic inside the channel.

4. Avoidance of duplication

```
protected String send(String preparedMessage) {
    return implementation.send(id, preparedMessage);
}
```

The base class keeps the common delegation step in one place. Reminder and UrgentAlert do not repeat the ID lookup or the call to the stored channel.

5. Encapsulation of state

```
private final String id;
private final String message;
private Channel implementation;

public void setImplementation(Channel implementation) {
    this.implementation = implementation;
}
```

The ID and message cannot be replaced after construction. The channel is private but can be changed through one public method, which is the operation used in T5.

Independent extension

The first version contained EmailChannel and SmsChannel and passed T1 through T5. Its base commit is 5e902fb6be0726e8e82c5097667815f71f66c352. PushChannel was then added as another implementation of the existing Channel interface. Main was updated to run T6 and T7, and sources.txt was updated so javac includes the new file.

Notification, Reminder, UrgentAlert, Channel, EmailChannel, and SmsChannel stayed unchanged. The file extension.diff records only Main.java and the new PushChannel.java within src. This is the independent extension required by the task.

Runtime switch

T5 keeps references named original and activeReference. The comparison original == activeReference proves they refer to the same Reminder object. After setImplementation(sms), the second execute() call produces SMS output. The stored ID and message are compared with their original values and remain unchanged.

Bridge and Adapter

Bridge separates two dimensions by design so both can vary independently. In this project, notification behavior and delivery format are developed through separate hierarchies joined by a Channel reference. Adapter has a different intent. It wraps an existing class so code expecting another interface can use it. Adapter usually solves an interface mismatch, while this Bridge defines the relationship before those hierarchies are extended.

Trade-off

The design adds more classes and one extra delegation step. A reader must follow a call from Reminder or UrgentAlert into Notification and then into the active Channel. That extra structure is useful here because new channels can be added without editing the notification hierarchy.

Submission evidence

The standard Java 17 commands compile the files listed in sources.txt. The captured run in demo-output.txt contains T1 through T7 and ends with SUMMARY: 7/7 PASS. The repository is https://github.com/iuve1n/SDP_Assignment3. The documented implementation commit is 4b8f3faf16ab9918e1675f817bfebd2ab6689e32.

References

- [1] Bekenov, Y. Assignment 3: Bridge Pattern. Astana IT University, 2026-2027.
- [2] Gamma, E., Helm, R., Johnson, R., and Vlissides, J. Design Patterns: Elements of Reusable Object-Oriented Software. Addison-Wesley, 1994.
- [3] PlantUML. Class Diagram Syntax. <https://plantuml.com/class-diagram>